Université — Département —

Cours / Module d'intelligence artificielle et vision

VisioFind

Recherche visuelle par apprentissage profond

Objets, animaux, lieux Images, vidéo et cartographie

Rapport de projet

Présenté par : Majda Rane, Ayoub Elkarkoub

Encadrant : Anouar Ben-Loghfery

Année universitaire 2026

Remerciements

Nous adressons nos remerciements à Anouar Ben-Loghfery pour l'encadrement, les orientations méthodologiques et les retours critiques tout au long de ce travail. Nous remercions également les enseignants du département pour les enseignements en traitement du signal, en vision par ordinateur et en apprentissage profond qui ont constitué les bases théoriques de ce projet. Enfin, nous salutations la communauté scientifique et open source (OpenAI, Hugging Face, les mainteneurs de Streamlit et PyTorch) sans laquelle la réalisation d'un tel prototype serait nettement plus coûteuse en temps et en ressources.

Résumé

Mots-clés : recherche visuelle, apprentissage profond, CLIP, traitement d'image, vidéo, similarité, Streamlit, Python.

Ce rapport présente **VisioFind**, une application interactive de *recherche d'images par le contenu* (*content-based image retrieval*, CBIR). L'utilisateur soumet une photographie ou une vidéo ; dans le second cas, le système extrait un ensemble d'images-clés (*frames*) parmi lesquelles l'utilisateur en sélectionne une pour l'analyse. Chaque image est projetée dans un espace de représentation de dimension 512 à l'aide du modèle pré-entraîné **CLIP** (`openai/clip-vit-base-patch32`), sans phase d'entraînement locale des poids du réseau. La similarité entre la requête et une galerie de référence organisée en dossiers (`animaux`, `objets_lieux`) est mesurée par le **cosinus** entre vecteurs normalisés. Une interface web **Streamlit** orchestre le chargement des données, la mise en cache de l'index sur disque et l'affichage des résultats. Pour certaines étiquettes de lieux, une intégration avec **Google Maps** propose un lien géographique illustratif. Le document détaille l'état de l'art, l'architecture logicielle, la constitution des jeux de données, les contraintes matérielles et les perspectives d'amélioration (indexation approximative, fine-tuning, déploiement).

Abstract

Keywords : visual search, deep learning, CLIP, image processing, video, similarity, Streamlit, Python.

This report describes **VisioFind**, an interactive *content-based image retrieval* (CBIR) application. The user submits a photograph or a video ; in the latter case, the system extracts key frames and lets the user pick one for analysis. Each image is embedded into a 512-dimensional space using the pre-trained **CLIP** model (`openai/clip-vit-base-patch32`), without on-site training of the network weights. Similarity to a reference gallery (folders `animaux` and `objets_lieux`) is measured by the **cosine** similarity between ℓ_2-normalized vectors. A **Streamlit** web interface orchestrates data loading, on-disk index caching, and result display. For selected place labels, **Google Maps** links provide illustrative geolocation. We discuss related work, software architecture, dataset construction, hardware constraints, and future directions (approximate indexing, fine-tuning, deployment).

Table des matières

Remerciements **1**

Résumé **2**

Abstract **3**

1 Introduction générale **8**
1.1 Contexte 8
1.2 Problématique 8
1.3 Objectifs du projet 8

2 État de l'art **10**
2.1 Deep learning : définition et importance 10
2.2 Traitement d'image 10
2.3 Applications dans la vie réelle 10
2.4 Exemples de systèmes et de familles de modèles 11
2.4.1 Réseaux convolutionnels et architectures classiques 11
2.4.2 Détection et segmentation 11
2.4.3 Vision Transformers et modèles multimodaux 11
2.4.4 TensorFlow *versus* PyTorch 11

3 Présentation du projet VisioFind **12**
3.1 Description générale 12
3.2 Fonctionnalités principales 12
3.2.1 Recherche d'images objets, animaux, lieux 12
3.2.2 Recherche vidéo analyse par images-clés 12
3.2.3 Intégration de cartes pour les lieux 13

4 Technologies utilisées **14**
4.1 Langages 14
4.2 Frameworks et environnement d'exécution 14
4.3 Bibliothèques et rôle détaillé 14

5 Modèles utilisés **16**
5.1 Modèle pré-entraîné retenu 16

5.2 Pourquoi ce modèle a été choisi 16
5.3 Principe de fonctionnement 16

6 Dataset **17**
6.1 Description des jeux de données 17
6.2 Sources des données 17
6.3 Justification des choix 17
6.4 Problèmes liés à la taille des datasets 18

7 Architecture du système **19**
7.1 Vue d'ensemble 19
7.2 Pipeline du traitement d'image et d'embedding 19
7.3 Fonctionnement global 20

8 Implémentation **21**
8.1 Étapes de développement 21
8.2 Explication conceptuelle du code 21
8.3 Interface utilisateur 22

9 Partie exécution interface et résultats **23**
9.1 Préparation et lancement 23
9.2 Mode animaux 23
9.3 Mode objets et lieux 25

10 Recherche vidéo **27**
10.1 Motivation de la fonctionnalité 27
10.2 Principe extraction des frames 27
10.3 Importance dans la vie réelle 27

11 Applications dans la vie quotidienne **28**

12 Contraintes et difficultés **29**
12.1 Problèmes matériels 29
12.2 Dataset volumineux 29
12.3 Temps de calcul 29
12.4 Limitations techniques diverses 29

13 Limites du projet **30**
13.1 Précision 30
13.2 Performance 30
13.3 Généralisation 30

14 Perspectives d'amélioration **31**

15 Conclusion générale **32**

A Guide d'installation, de configuration et d'exécution **33**
A.1 Environnement Python 33
A.2 Lancement de l'application 33
A.3 Constitution initiale de la galerie 33
A.4 Authentification Kaggle (optionnel) 34

B Compléments techniques et éthiques **35**
B.1 Métriques d'évaluation en CBIR 35
B.2 Indexation approximative 35
B.3 Données, vie privée et propriété intellectuelle 35
B.4 Pistes de reproductibilité expérimentale 35

Bibliographie **36**

Liste des figures

7.1 Architecture logique de VisioFind : l'interface orchestre l'encodage, l'index (avec persistance), et l'affichage conditionnel des liens cartographiques. 19
7.2 Pipeline séquentiel de la recherche par similarité dans VisioFind. 20

9.1 Interface barre latérale en mode *Animaux* : choix du type de recherche, nombre de résultats (`top_k`) et option de réindexation forcée. 23
9.2 Étape d'upload téléversement de l'image de requête et aperçu avant lancement de la recherche visuelle. 24
9.3 Résultats mode *Animaux* : grille des images les plus similaires avec scores (pourcentage) et noms de classes issus de la structure de dossiers. 24
9.4 Interface passage au mode *Objets et lieux* pour interroger la seconde galerie de référence. 25
9.5 Upload image de requête représentative d'objets ou de scènes / lieux. 26
9.6 Résultats similarité visuelle dans la galerie `objets_lieux` (scores et étiquettes de dossiers). 26
9.7 Intégration cartographique lorsque l'étiquette du résultat figure dans `KNOWN_PLACES`, affichage du lieu et lien *Google Maps*. 26

Chapitre 1

Introduction générale

1.1 Contexte

L'explosion du volume d'images et de vidéos numériques réseaux sociaux, smartphones, caméras connectées, bases documentaires rend indispensable le développement d'outils capables de **comprendre** et **organiser** le contenu visuel au-delà de simples noms de fichiers ou balises textuelles saisies à la main. Parallèlement, l'**apprentissage profond** a transformé la vision par ordinateur : les réseaux convolutionnels puis les *Vision Transformers* permettent d'extraire des représentations denses, robustes partiellement aux variations d'éclairage, d'échelle ou de cadrage.

Dans un cadre pédagogique et de prototypage, il est souvent irréaliste d'entraîner de toute pièce un modèle de grande taille sur un cluster de calcul. L'usage de **modèles pré-entraînés** entraînés par des acteurs industriels ou académiques sur des corpus massifs puis leur **réutilisation en inférence** pour des tâches downstream (recherche, tri, annotation assistée) constitue une approche pragmatique et largement adoptée.

1.2 Problématique

La recherche d'images par **mots-clés** suppose que l'utilisateur dispose d'un vocabulaire adéquat et partagé avec celui du système d'indexation. Or :

— il est fréquent de posséder une **image** sans savoir la décrire précisément (espèce animale, type d'architecture, objet rare) ;

— l'ambiguïté linguistique (polysemy, traduction) complique l'interrogation textuelle ;

— l'utilisateur peut chercher une **ressemblance visuelle globale** (style, composition) plutôt qu'une étiquette discrète.

La **recherche par similarité dans un espace d'embeddings** aprend par le contenu visuel lui-même. Elle ne remplace pas une base de métadonnées riche, mais la **complète** utilement lorsque la scène précède le langage.

1.3 Objectifs du projet

Les objectifs de **VisioFind** sont les suivants :

1. Concevoir une application **accessible** (interface web) permettant la recherche visuelle dans une galerie locale.
2. Distinguer deux **contextes de recherche** animaux d'une part, objets et lieux d'autre part afin de limiter les confusions inter-domaines.
3. Intégrer le support des **vidéos** par extraction d'images-clés et **sélection utilisateur** de la frame analysée.
4. Associer, pour certaines classes de lieux, un **lien cartographique** (Google Maps) à titre d'enrichissement contextuel.
5. Mettre en œuvre un **cache d'index** pour réduire le temps de démarrage lors des sessions successives.
6. Documenter de manière **rigoureuse** les choix techniques, les limites et les pistes d'évolution pour un public universitaire.

Le périmètre exclut explicitement l'entraînement complet de CLIP sur le site utilisateur ; les poids sont **gelés**. Le périmètre inclut en revanche la **constitution** et la **mise à jour** de la galerie de référence par scripts automatisés.

Chapitre 2

État de l'art

2.1 Deep learning : définition et importance

L'**apprentissage profond** (*deep learning*) désigne une famille de méthodes d'apprentissage automatique fondées sur des modèles paramétriques composés de **plusieurs couches** de transformations non linéaires (réseaux de neurones profonds). Contrairement à l'ingénierie de caractéristiques manuelle classique (filtres de Gabor, SIFT, HOG, etc.), le réseau **apprend** des représentations hiérarchiques à partir des données et d'une fonction de perte.

L'**importance** du deep learning en vision tient à plusieurs facteurs : capacité de généralisation sur des données haute dimension, disponibilité de **gros corpus** annotés ou faiblement structurés, accélération matérielle (GPU, TPU), et architectures de plus en plus efficaces. Pour la **recherche d'images**, l'enjeu est de mapper les images dans un espace où la **distance** (ou la similarité) reflète la proximité **sémantique** ou **perceptive** souhaitée.

2.2 Traitement d'image

Le **traitement d'image** couvre l'acquisition, le prétraitement (correction, filtrage, redimensionnement), parfois la segmentation ou la détection d'objets, jusqu'à l'extraction de descripteurs ou la classification. Dans VisioFind, les étapes **élémentaires** incluent :

— lecture des fichiers (`jpg`, `png`, `webp`) et conversion en **RGB** ;

— pour la vidéo, **décodage** et **échantillonnage temporel** des frames via OpenCV ;

— **normalisation** et redimensionnement conformes au `CLIPProcessor` (pipeline Hugging Face).

Il n'y a pas, dans la version actuelle, de **segmentation sémantique** ni de découpe d'objet en amont : l'embedding porte sur **l'image entière**.

2.3 Applications dans la vie réelle

Des applications proches existent dans : la **recherche inverse** en commerce électronique (ń trouver un produit visuellement similaire ż), la **modération** de contenus, l'**archivage** de collections patrimoniales, l'assistance à l'**annotation** médicale (sous validation clinique stricte), la **surveillance** intelligente (avec enjeux éthiques majeurs), ou encore les moteurs de recherche

multimodaux. VisioFind se situe au niveau **pédagogique et prototype** : galerie locale, transparence du code, pas de dépendance obligatoire à un cloud propriétaire pour l'inférence.

2.4 Exemples de systèmes et de familles de modèles

2.4.1 Réseaux convolutionnels et architectures classiques

Les **CNN** (*Convolutional Neural Networks*) ont dominé la classification d'images pendant une décennie. Des architectures telles que **ResNet** (connexions résiduelles, très profondes) ou **EfficientNet** (compromis précision / coût) produisent des descripteurs utiles lorsqu'on retire la dernière couche de classification. Ils sont souvent entraînés ou affinés sur **ImageNet** pour la classification en 1000 classes.

2.4.2 Détection et segmentation

YOLO, **R-CNN** et dérivés visent la **localisation** d'objets (boîtes englobantes) ou la classification pixel à pixel. Ils répondent à d'autres besoins que la CBIR globale ; une évolution de VisioFind pourrait combiner détection locale + embedding de régions.

2.4.3 Vision Transformers et modèles multimodaux

Les **Vision Transformers** (ViT) découpent l'image en patchs et appliquent des mécanismes d'attention analogues au traitement du langage. **CLIP** (Contrastive LanguageImage Pretraining) apprend conjointement des représentations d'images et de textes alignées par contraste sur des millions de paires tirées du Web. Cette alignement multimodal permet, entre autres, la **recherche zero-shot** par texte ; ici, nous n'exploitons que la **branche image** pour comparer des images entre elles.

2.4.4 TensorFlow *versus* PyTorch

TensorFlow et **PyTorch** sont les deux écosystèmes dominants. TensorFlow offre historiquement de bons outils de déploiement (TensorFlow Lite, Serving) ; PyTorch est très répandu en recherche pour sa flexibilité. Le hub **Hugging Face** s'intègre naturellement à PyTorch pour CLIP ; c'est pourquoi **VisioFind adopte PyTorch** pour l'inférence, sans utiliser TensorFlow dans l'implémentation décrite.

Chapitre 3

Présentation du projet VisioFind

3.1 Description générale

VisioFind est une application **locale** écrite en Python. Elle parcourt récursivement les images sous `data/reference/<mode>/`, calcule ou recharge les **embeddings** CLIP, et répond aux requêtes utilisateur par un classement des k plus proches voisins en similarité cosinus. L'interface est servie par **Streamlit** (protocole HTTP local par défaut).

3.2 Fonctionnalités principales

3.2.1 Recherche d'images objets, animaux, lieux

Deux modes sont exposés dans la barre latérale :

`animaux`
Galerie structurée par sous-dossiers (espèces ou familles). Les étiquettes affichées correspondent au **nom du dossier parent**.

`objets_lieux`
Objets du quotidien et scènes (plage, rue, montagne, parc, restaurant, etc.). Même principe d'étiquetage par structure de fichiers.

Il ne s'agit pas d'un classifieur entraîné sur ces classes dans le dépôt : CLIP fournit un espace continu ; le **rôle des dossiers** est de **segmenter** le corpus et d'attribuer des **légendes lisibles** aux résultats.

3.2.2 Recherche vidéo analyse par images-clés

L'utilisateur peut téléverser une vidéo (`mp4`, `mov`, `avi`, `mkv`). Le module `extract_video_frames` échantillonne une frame tous les n indices de trame (par défaut $n = 15$), jusqu'à un maximum de 24 images exportées. L'utilisateur **choisit explicitement** quelle frame sert de requête. Ce choix est délibéré : une vidéo est une **séquence** ; une seule image automatique pourrait être non représentative.

3.2.3 Intégration de cartes pour les lieux

Lorsque le libellé d'un résultat correspond à une entrée du dictionnaire `KNOWN_PLACES` dans `src/places.py`, l'interface affiche un texte du type ń Lieu détecté ż et un hyperlien **Google Maps** construit à partir de coordonnées (latitude, longitude) et d'un nom d'affichage. Il s'agit d'une **mise en contexte pédagogique** : les coordonnées sont **associées à la classe**, non inférées par géolocalisation EXIF ou GPS de la photo utilisateur.

Chapitre 4

Technologies utilisées

4.1 Langages

Python 3 est le langage unique du projet applicatif. Il permet l'intégration rapide de bibliothèques scientifiques, l'orchestration de Streamlit et l'appel aux API Hugging Face. Les annotations de type (`typing`) améliorent la lisibilité du code.

4.2 Frameworks et environnement d'exécution

— **Streamlit** : framework d'applications de données et de prototypes ML ; widgets déclaratifs, mise en cache (`@st.cache_resource`) pour l'index.

— **PyTorch** : moteur de tenseurs et d'automatisation de la différenciation ; ici utilisé en **mode inférence** avec `torch.no_grad()`.

— **Hugging Face Transformers** : abstraction de haut niveau pour `CLIPModel` et `CLIPProcessor`.

— **OpenCV** : lecture vidéo et écriture des frames sur disque temporaire.

4.3 Bibliothèques et rôle détaillé

Le tableau 4.1 résume les dépendances principales déclarées dans `requirements.txt`.

Table 4.1 – Bibliothèques Python et rôle dans VisioFind.

Paquet	**Rôle**
`streamlit`	Interface web, upload, affichage, cache de ressources.
`torch`, `torchvision`	Inférence CLIP, écosystème PyTorch.
`transformers`	Chargement du modèle et du processeur CLIP.
`Pillow`	I/O images, conversion RGB.
`opencv-python`	Décodage vidéo, export des frames.
`numpy`	Matrices d'embeddings, produits scalaires, tri, fichiers `.npz`.
`ddgs`, `icrawler`, `requests`	Collecte d'images (scripts de dataset).
`tqdm`	Progression dans les scripts.
`kaggle`	Téléchargement via API Kaggle.
`pandas`	Analyses ou scripts auxiliaires (optionnel pour le cur CBIR).

Chapitre 5

Modèles utilisés

5.1 Modèle pré-entraîné retenu

Le modèle utilisé est **CLIP ViT-B/32**, identifiant Hugging Face `openai/clip-vit-base-patch32`. Il combine :

— un encodeur **image** basé sur un **Vision Transformer** avec patchs de taille 32×32 ;

— un encodeur **texte** (non utilisé pour la similarité imageimage dans cette implémentation) ;

— une couche de projection vers un espace commun de dimension **512**.

5.2 Pourquoi ce modèle a été choisi

— **Pré-entraînement massif** : représentations généralistes riches, sans entraînement local coûteux.

— **Compatibilité Hugging Face** : installation et mise à jour simples via `from_pretrained`.

— **Compromis taille / performance** : la variante `base` reste raisonnable pour des machines étudiantes en CPU.

— **Adequation à la CBIR** : les embeddings image sont comparables par produit scalaire après normalisation ℓ_2.

5.3 Principe de fonctionnement

Soit f_θ l'encodeur image CLIP et x une image prétraitée. On obtient $v = f_\theta(x) \in \mathbb{R}^{512}$, puis $\hat{v} = v/\|v\|_2$. Pour une galerie de N images avec vecteurs normalisés $\hat{v}_1, \ldots, \hat{v}_N$, le score de similarité cosinus entre la requête $\hat{q}$ et l'image i est $\hat{q}^\top \hat{v}_i \in [-1, 1]$. L'implémentation calcule le produit matricevecteur $\mathbf{V}\hat{q}$ où les lignes de $\mathbf{V}$ sont les $\hat{v}_i^\top$, puis trie les indices par score décroissant pour retourner les `top_k`.

Remarque. Des modèles tels que **ResNet** ou **YOLO** sont pertinents en classification ou détection ; ils ne sont **pas** le moteur principal de VisioFind, qui repose sur **CLIP**. Ils demeurent cependant des références incontournables de l'état de l'art, d'où leur mention au chapitre 2.

Chapitre 6

Dataset

6.1 Description des jeux de données

On distingue deux niveaux :

(A) Données de pré-entraînement de CLIP. Elles ne sont pas stockées dans le dépôt étudiant. OpenAI a entraîné CLIP sur un très grand nombre de paires (image, texte) filtrées depuis Internet. Ce pré-entraînement confère au modèle une **géométrie de l'espace des embeddings** favorable à la proximité sémantique entre images (et textes) similaires.

(B) Galerie de référence du projet. Elle réside sous `data/reference/animaux/` et `data/reference/obje`
Chaque **sous-dossier** correspond à une classe logique ; chaque fichier image valide est une **occurrence** de cette classe.

6.2 Sources des données

— **Script web** `scripts/build_dataset.py` : recherche d'images via `ddgs` (DuckDuckGo), avec **repli** sur Bing Images via `icrawler` si le réseau bloque le premier fournisseur.

— **Script Kaggle** `scripts/build_dataset_kaggle.py` : téléchargement de jeux publics lorsque la collecte web est instable ; authentification par fichier `kaggle.json` (voir `scripts/check_kaggle_auth.py`

— **Ajout manuel** : copie d'images dans les dossiers appropriés pour enrichir ou corriger le corpus.

6.3 Justification des choix

La diversité des sources vise la **résilience** (pannes réseau, quotas) et la **couverture thématique** (animaux, objets, paysages). L'organisation par dossiers impose une **taxonomie contrôlée** par les auteurs du projet, indépendamment des logits texteimage de CLIP.

6.4 Problèmes liés à la taille des datasets

— **Temps d'indexation** : chaque image nécessite un passage avant CLIP ; un corpus de plusieurs dizaines de milliers d'images devient pénalisant en CPU.

— **Stockage disque** : images sources + fichiers de cache `.npz` (matrice $N \times 512$ en `float32`).

— **Déséquilibre de classes** : si un dossier contient beaucoup plus d'images qu'un autre, il peut **saturer** les résultats du top-k même lorsque la similarité marginale est faible.

— **Qualité et droits** : images web bruitées, watermarks, ou soumises à des licences restrictives problème éthique et légal à mentionner dans tout rapport académique sérieux.

Chapitre 7

Architecture du système

7.1 Vue d'ensemble

La figure 7.1 résume les composants logiciels et le flux de données.

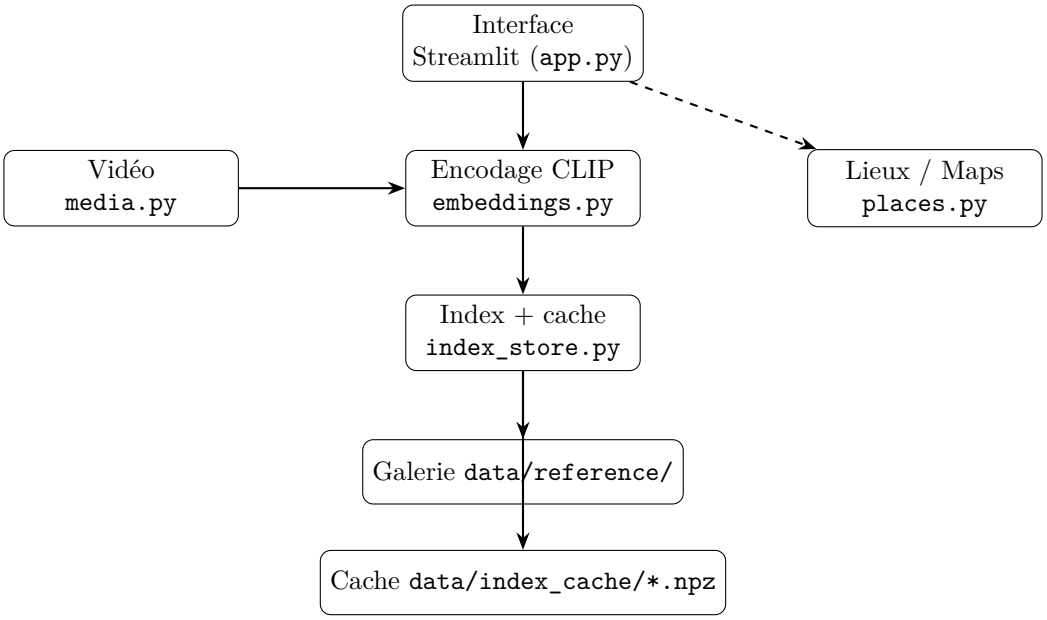

Figure 7.1 – Architecture logique de VisioFind : l'interface orchestre l'encodage, l'index (avec persistance), et l'affichage conditionnel des liens cartographiques.

7.2 Pipeline du traitement d'image et d'embedding

1. **Sélection du mode** (`animaux` ou `objets_lieux`) et chargement de l'index depuis le cache si la empreinte du dataset est inchangée.
2. **Upload** : si image, conversion RGB ; si vidéo, extraction des frames et choix utilisateur.
3. **Prétraitement CLIP** : redimensionnement, normalisation (géré par le `CLIPProcessor`).
4. **Inférence** : `get_image_features` avec gestion des versions de l'API (tensor brut ou objet avec `pooler_output`).
5. **Normalisation** ℓ_2 des vecteurs requête.

6. **Scoring** : produit scalaire avec la matrice d'index, tri, troncature à k.
7. **Rendu** : grille d'images, scores en pourcentage (cosinus $\times 100$), liens Maps si étiquette connue.

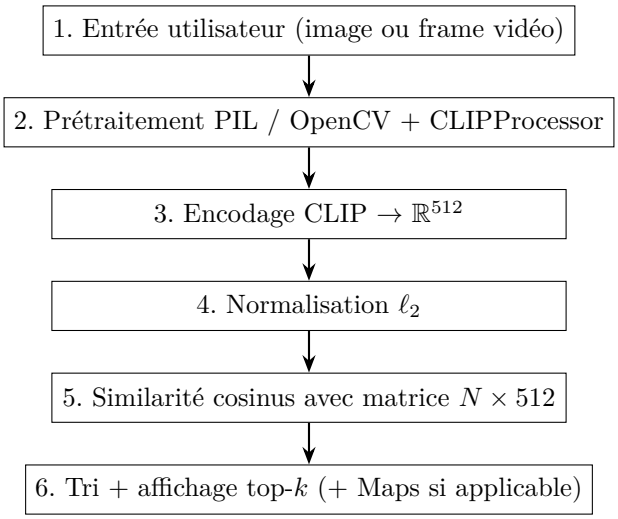

Figure 7.2 – Pipeline séquentiel de la recherche par similarité dans VisioFind.

7.3 Fonctionnement global

Le module `index_store.py` parcourt récursivement les chemins, ignore les extensions non supportées, charge les images avec Pillow, appelle `embed_images` par batch implicite (toutes les images en mémoire lors du build limite pour très gros corpus). Une **signature** du dataset (nombre de fichiers, *mtime* maximal) invalide le cache si la galerie change. La recherche `search_similar` effectue un **scan linéaire** $\mathcal{O}(N)$ par requête.

Chapitre 8

Implémentation

8.1 Étapes de développement

1. Définition du périmètre fonctionnel (modes, vidéo, cache, Maps).
2. Prototypage de l'encodage CLIP et validation des dimensions de sortie.
3. Implémentation de l'index persistant et des tests manuels sur petit corpus.
4. Développement de l'interface Streamlit et branchement des paramètres (`top_k`, réindexation forcée).
5. Scripts de collecte de données et documentation utilisateur (`README.md`).
6. Campagne de captures d'écran et rédaction du rapport.

8.2 Explication conceptuelle du code

`src/embeddings.py`. Fonction `get_clip()` mise en cache (`lru_cache`) pour ne charger le modèle qu'une fois. La fonction `make_square` padde avec du noir pour obtenir une image carrée avant traitement. `embed_images` applique le processeur, extrait les features, normalise.

`src/index_store.py`. `list_images` collecte les chemins ; `build_index` calcule la matrice ; `build_index_cached` lit ou écrit un fichier `.npz` compressé.

`src/media.py`. Boucle de lecture OpenCV, sauvegarde espacée des frames.

`app.py`. Point d'entrée Streamlit : radio pour le mode, slider `top_k`, case à cocher de réindexation, `file_uploader`, branchement vidéo/image, bouton de recherche, affichage via `show_results`.

Listing 8.1 – Schéma simplifié de la boucle de recherche (conceptuel).

```
# Pseudo-code proche de la logique applicative
vectors, paths, labels = build_index_cached(root, cache_npz, force)
q = embed_images([query_image])[0] # vecteur normalisé
scores = vectors @ q # produits scalaires = cosinus
top_indices = argsort(-scores)[:k]
```

8.3 Interface utilisateur

L'interface privilégie la **simplicité** : barre latérale pour les hyperparamètres, zone centrale pour l'aperçu de la requête et la grille de résultats. Les pourcentages affichés sont une **transposition pédagogique** du cosinus (valeur en $[-1, 1]$ ramenée à un pourcentage pour la lisibilité). Le bouton **Forcer réindexation** vide le cache Streamlit et reconstruit l'index lorsque la galerie a été modifiée.

Chapitre 9

Partie exécution interface et résultats

9.1 Préparation et lancement

Après installation des dépendances (`pip install -r requirements.txt`) et constitution minimale de la galerie, la commande `streamlit run app.py` (ou `python -m streamlit run app.py` sous Windows si la commande `streamlit` n'est pas dans le `PATH`) démarre le serveur local. Les figures 9.1 à 9.7 illustrent deux scénarios : mode **Animaux**, puis mode **Objets et lieux** avec lien cartographique.

9.2 Mode animaux

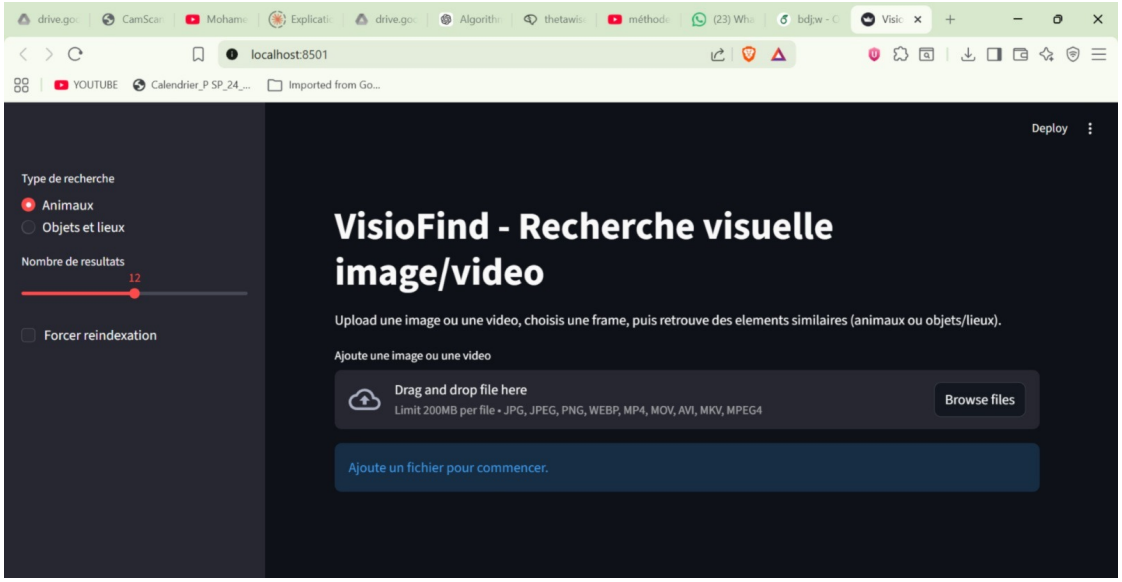

Figure 9.1 – Interface barre latérale en mode *Animaux* : choix du type de recherche, nombre de résultats (`top_k`) et option de réindexation forcée.

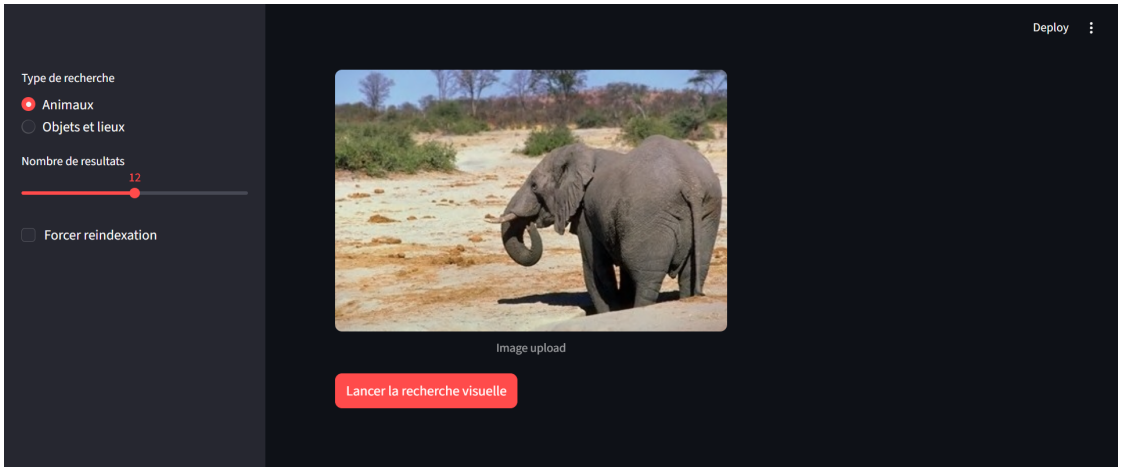

Figure 9.2 – Étape d'upload téléversement de l'image de requête et aperçu avant lancement de la recherche visuelle.

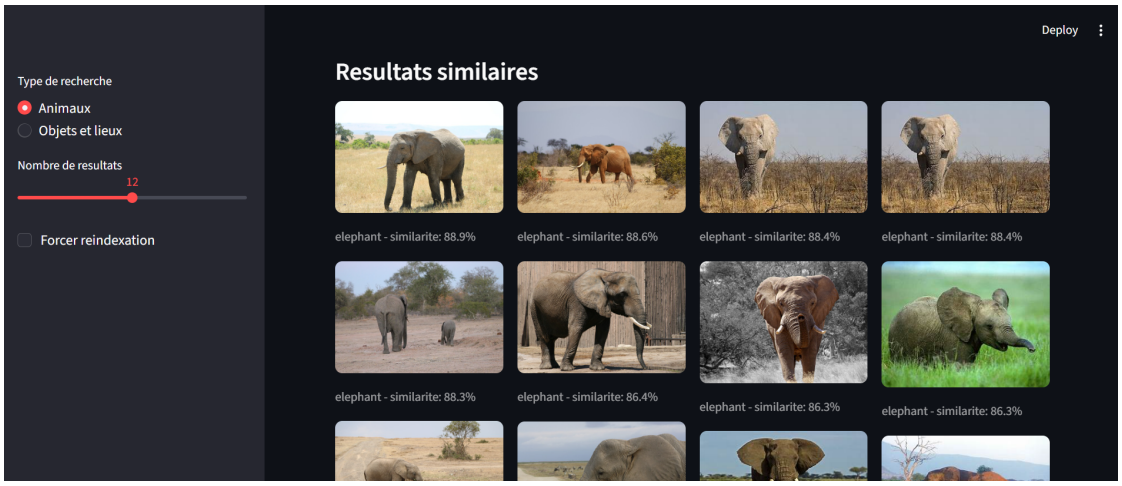

Figure 9.3 – Résultats mode *Animaux* : grille des images les plus similaires avec scores (pourcentage) et noms de classes issus de la structure de dossiers.

9.3 Mode objets et lieux

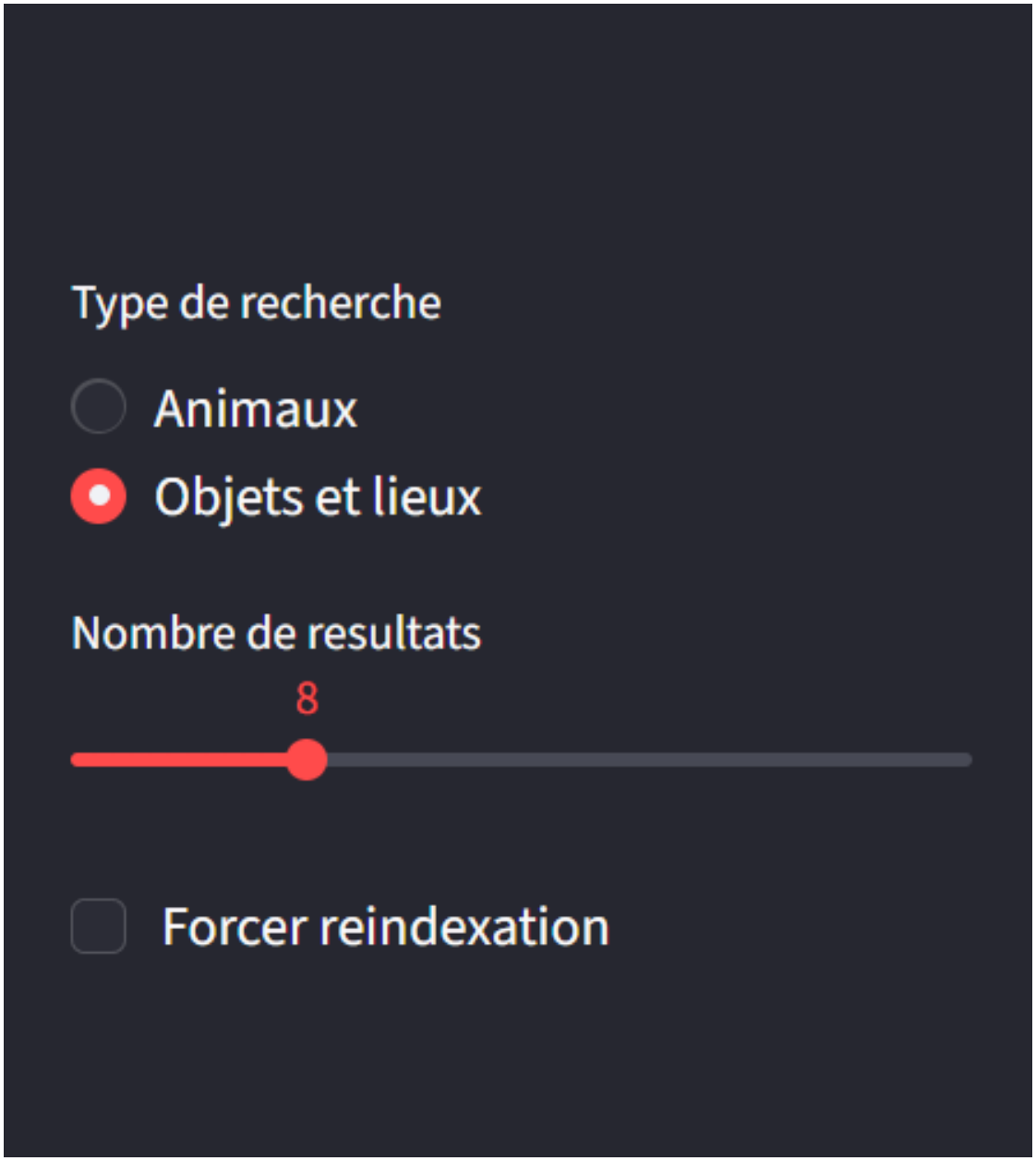

Figure 9.4 – Interface passage au mode *Objets et lieux* pour interroger la seconde galerie de référence.

Note méthodologique. Les captures proviennent du répertoire `docs/screenshots/` du projet. Le document accepte une compilation depuis `docs/` *ou* depuis la racine du projet grâce à plusieurs chemins déclarés dans `\graphicspath`.

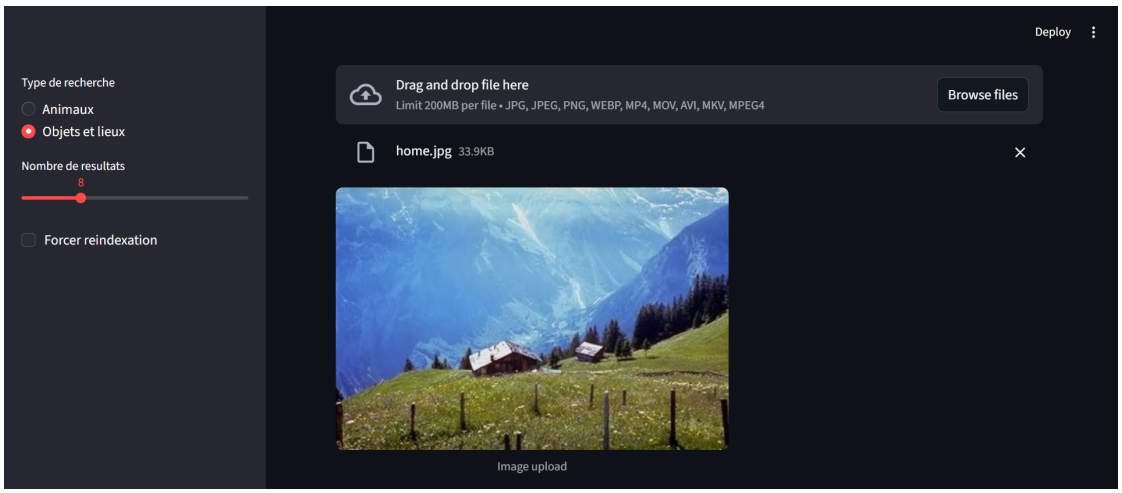

Figure 9.5 – Upload image de requête représentative d'objets ou de scènes / lieux.

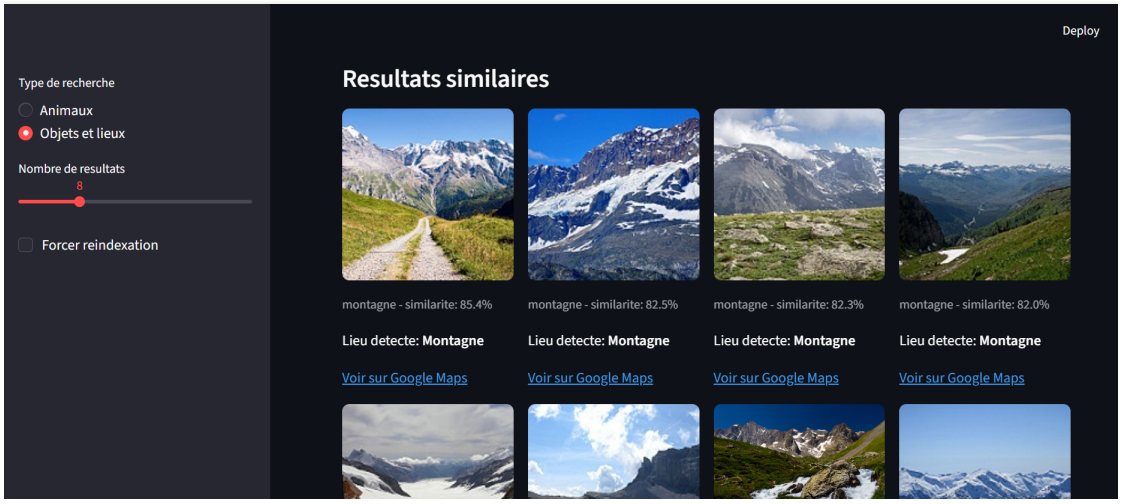

Figure 9.6 – Résultats similarité visuelle dans la galerie `objets_lieux` (scores et étiquettes de dossiers).

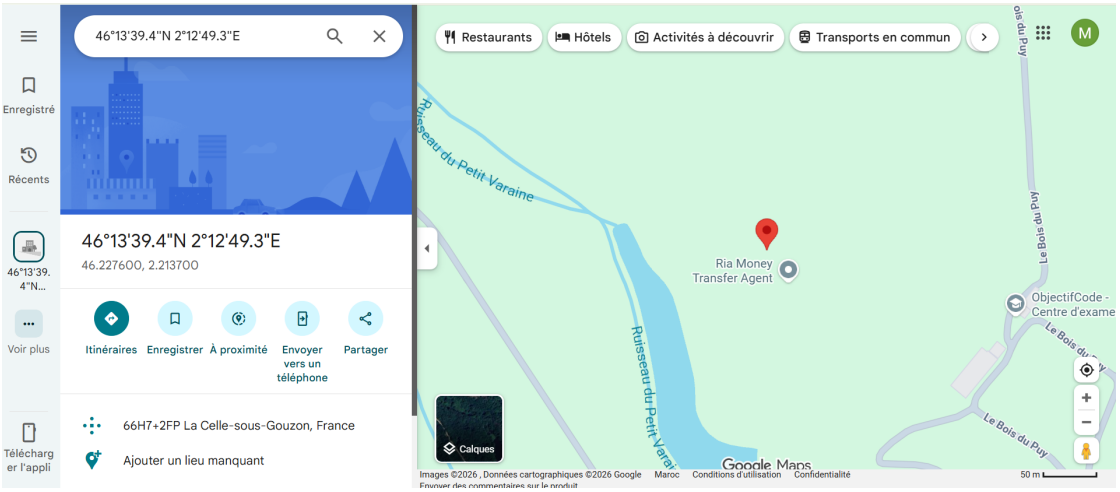

Figure 9.7 – Intégration cartographique lorsque l'étiquette du résultat figure dans `KNOWN_PLACES`, affichage du lieu et lien *Google Maps*.

Chapitre 10

Recherche vidéo

10.1 Motivation de la fonctionnalité

Une vidéo encode une **séquence temporelle** d'informations. Une frame isolée, choisie arbitrairement au début du clip, peut être floue, sous-exposée ou hors sujet. La fonctionnalité vidéo de VisioFind répond au besoin de **sélectionner l'instant pertinent** avant la recherche, ce qui rapproche le prototype des usages réels (archives touristiques, vidéos de surveillance amateur, contenus éducatifs).

10.2 Principe extraction des frames

OpenCV ouvre le flux vidéo, lit les trames séquentiellement, et enregistre une image sur disque tous les `every_n_frames` (valeur par défaut 15), jusqu'à `max_frames` (24). L'interface Streamlit liste les fichiers générés et un `selectbox` permet la sélection. Seule la frame choisie est passée au pipeline CLIP.

10.3 Importance dans la vie réelle

La navigation par *keyframes* est un paradigme classique en indexation vidéo. Pour un utilisateur non expert, choisir une image représentative est souvent plus simple que de formuler une requête textuelle sur le contenu dynamique. Les limites actuelles (échantillonnage régulier, pas d'analyse sémantique temporelle) ouvrent des perspectives de recherche (détection de plans, résumé vidéo).

Chapitre 11

Applications dans la vie quotidienne

— **Organisation de albums personnels** : retrouver des clichés ń ressemblant à cette photo ż sans étiquetage manuel exhaustif.

— **Commerce et design** : inspiration visuelle, recherche de produits analogues (dans une version industrielle couplée à un catalogue).

— **Éducation et musées** : médiation numérique, parcours guidés par similarité visuelle.

— **Prototypage recherche** : base de code pour expérimenter FAISS, fine-tuning ou multimodalité texteimage.

Chapitre 12

Contraintes et difficultés

12.1 Problèmes matériels

Les machines sans **GPU** subissent des temps d'inférence CLIP plus longs, surtout lors de la première construction d'index. La **RAM** limite le batch d'images chargées simultanément lors du `build_index`. Le **disque** doit accueillir le corpus et les caches.

12.2 Dataset volumineux

Au-delà de quelques milliers d'images, le scan linéaire et l'absence d'index vectoriel spécialisé (FAISS) deviennent des goulots d'étranglement. La maintenance d'un corpus hétérogène (doublons, images corrompues) demande des scripts de validation.

12.3 Temps de calcul

Le premier téléchargement des poids CLIP depuis Hugging Face dépend de la bande passante. Chaque session avec réindexation forcée reproduit le coût complet d'encodage.

12.4 Limitations techniques diverses

Blocages réseau (DuckDuckGo), configuration Kaggle sous Windows (fichier `kaggle.json` mal nommé), évolutions de l'API `transformers`, et comportement de Streamlit avec de très grandes images uploadées.

Chapitre 13

Limites du projet

13.1 Précision

Sans fine-tuning, CLIP peut confondre des classes fines (races, espèces proches) ou des scènes ambiguës. Les scores de cosinus ne sont pas des probabilités calibrées sur un jeu de labels du projet.

13.2 Performance

Complexité $\mathcal{O}(N)$ par requête en nombre d'images indexées. Pas de parallélisation multi-GPU ni de sharding des données.

13.3 Généralisation

Les biais du pré-entraînement (géographie, culture visuelle dominante) se répercutent sur les embeddings. Une galerie de référence déséquilibrée skew les résultats du top-k.

Chapitre 14

Perspectives d'amélioration

— **Indexation approximative** (FAISS, ScaNN) pour le passage à l'échelle.

— **Fine-tuning** contrastif ou adaptation légère (*adapter* layers) sur un domaine ciblé.

— **Segmentation / détection** préalable pour encoder des régions d'intérêt.

— **Géolocalisation réelle** à partir des métadonnées EXIF lorsqu'elles existent.

— **API REST** et conteneurisation Docker pour le déploiement.

— **Évaluation quantitative** (précision@k, protocole de requêtes annotées).

Chapitre 15

Conclusion générale

VisioFind illustre de bout en bout une chaîne moderne de **recherche visuelle par contenu** : constitution d'un corpus structuré, encodage par un modèle profond pré-entraîné multimodal (CLIP), persistance d'index, interface web interactive, extension vidéo par frames, et enrichissement cartographique pour des classes de lieux déclaratives. Le projet met en lumière les **atouts** des embeddings généralistes tout en rappelant les **limites** performance en CPU, absence de segmentation, biais de données et les **pistes** classiques d'un système de production (index rapide, spécialisation du modèle, évaluation rigoureuse). Sur le plan pédagogique, il constitue une passerelle tangible entre les concepts d'apprentissage profond, de traitement d'image et d'ingénierie logicielle web.

Annexe A

Guide d'installation, de configuration et d'exécution

A.1 Environnement Python

Il est fortement recommandé d'isoler les dépendances dans un environnement virtuel. Sous Windows, depuis la racine du projet :

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Si une ancienne dépendance `duckduckgo-search` était installée, la migration vers `ddgs` s'effectue par désinstallation puis réinstallation conformément au fichier `README.md`.

A.2 Lancement de l'application

```
streamlit run app.py
```

En cas d'absence de la commande `streamlit` dans le `PATH` (environnement virtuel non activé), utiliser :

```
python -m streamlit run app.py
```

A.3 Constitution initiale de la galerie

Avant une démonstration publique, exécuter par exemple :

```
python scripts/build_dataset.py --preset extended --per-class 120
```

Les images sont réparties sous `data/reference/animaux/` et `data/reference/objets_lieux/`. Après ajout manuel massif, cocher **Forcer réindexation** dans l'interface pour invalider le cache et recalculer les embeddings.

A.4 Authentification Kaggle (optionnel)

Placer le fichier `kaggle.json` dans le dossier caché `.kaggle` du répertoire personnel Windows (par exemple `C:\Users\NomUtilisateur\.kaggle\`), en veillant à ce que le fichier ne soit pas enregistré sous le nom erroné `kaggle.json.txt`. La commande `python scripts/check_kaggle_auth.py` permet un diagnostic rapide.

Annexe B

Compléments techniques et éthiques

B.1 Métriques d'évaluation en CBIR

En recherche académique, on évalue souvent un système CBIR par la **précision@k** (proportion de résultats pertinents parmi les k premiers) ou le **rappel** sur un jeu de requêtes pour lequel la vérité terrain (images pertinentes) a été annotée. VisioFind n'implémente pas encore ce protocole ; son intégration constitue une piste d'amélioration méthodologique majeure.

B.2 Indexation approximative

Lorsque N dépasse le million de vecteurs, le produit scalaire exhaustif devient coûteux. Des bibliothèques comme **FAISS** (Facebook AI Similarity Search) proposent des structures de données (HNSW, IVF) qui échangent une légère imprécision contre un gain de temps dramatique. La migration consisterait à sérialiser les embeddings au format attendu par FAISS et à remplacer l'appel `np.dot` + `argsort` par une recherche de voisins approximatifs.

B.3 Données, vie privée et propriété intellectuelle

La collecte automatisée d'images depuis le Web soulève des questions de **copyright** et de **consentement**. Pour un usage institutionnel, il convient de privilégier des corpus sous licence ouverte, d'obtenir des autorisations, ou de produire ses propres photographies. Côté vie privée, tout déploiement traitant des données personnelles devrait respecter le RGPD et les politiques locales (anonymisation, durée de conservation, droit à l'effacement).

B.4 Pistes de reproductibilité expérimentale

Pour renforcer la rigueur scientifique du projet, on peut figer les versions exactes des paquets (`pip freeze`), documenter le matériel (CPU, RAM, présence GPU), et journaliser les temps d'indexation et de requête en fonction de N et de k.

Bibliographie

[1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I. *Learning Transferable Visual Models From Natural Language Supervision.* arXiv :2103.00020, 2021. https://arxiv.org/abs/2103.00020

[2] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N. *An Image is Worth 16x16 Words : Transformers for Image Recognition at Scale.* arXiv :2010.11929, 2020.

[3] He, K., Zhang, X., Ren, S., Sun, J. *Deep Residual Learning for Image Recognition.* Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[4] Redmon, J., Divvala, S., Girshick, R., Farhadi, A. *You Only Look Once : Unified, Real-Time Object Detection.* Proceedings of CVPR, 2016.

[5] Smeulders, A. W. M., Worring, M., Santini, S., Gupta, A., Jain, R. *Content-Based Image Retrieval at the End of the Early Years.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 22(12), 2000.

[6] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L. *ImageNet : A Large-Scale Hierarchical Image Database.* Proceedings of CVPR, 2009.

[7] Wolf, T. et al. *Transformers : State-of-the-Art Natural Language Processing.* Proceedings of EMNLP (demonstration), 2020. Documentation CLIP : https://huggingface.co/docs/transformers

[8] Streamlit Inc. *Streamlit A faster way to build and share data apps.* https://streamlit.io/

[9] Paszke, A. et al. *PyTorch : An Imperative Style, High-Performance Deep Learning Library.* Advances in Neural Information Processing Systems (NeurIPS), 2019.